Korean IGT - Experimental Protocol V3

Aligned with the 17 Nov agenda and the updated task specification. Team baseline; experimental hyperparameters are proposals, not confirmed optimal settings.

1. Models and task definition

• **Models:** Bolmo 1B and Bolmo 7B (confirm exact available model IDs and hardware requirements before running).

• **Language:** Korean only. Use the corrected Korean IGT dataset once validated.

• **Input (both approaches):** original Korean script + its English translation.

• **Output:** predicted transliteration/transcription + morphological gloss.

• **Important:** the model must predict transliteration; gold transliteration must never be supplied at inference.

Dataset field mapping (provisional until team verifies the rebuilt dataset): original_script / text -> input; translation -> input; transcription/transliteration -> gold target 1; glosses -> gold target 2. Preserve terminology defined by the data annotation guidelines.

2. Zero-shot prompts (no fine-tuning)

A. All-in-one: one model call

```
You are a linguist specializing in interlinear glossed text (IGT).
Language: Korean
Original Korean script: {original_script}
English translation: {translation}

Predict (1) the transliteration of the Korean script and
(2) the corresponding morpheme-level gloss.
Preserve word and morpheme order and use consistent IGT labels.
Do not explain your answer.

Transliteration: ...
Gloss: ...
```

B. Step-by-step: two model calls

Step 1 prompt - predict transliteration:

```
You are a linguist specializing in Korean transcription.
Original Korean script: {original_script}
English translation: {translation}

Predict the transliteration of the Korean script.
Do not output a gloss or explanation.
Transliteration:
```

Step 2 prompt - predict gloss using Step 1 output:

```
You are a linguist specializing in interlinear glossing.
Original Korean script: {original_script}
English translation: {translation}
Predicted transliteration: {predicted_transliteration}

Predict the morpheme-level gloss corresponding to this text.
Do not explain your answer.
Gloss:
```

Execution: apply both prompting methods to the exact same held-out test items for both model sizes. Set do_sample=False; use the same maximum generation budget and record raw outputs. The Step 2 input must contain the model-predicted, not gold, transliteration.

3. LoRA fine-tuning - step-by-step

Use one adapter for two supervised task types as the first baseline. Training examples may use gold transliteration as teacher-forced input for gloss learning; validation/test inference must use model-predicted transliteration. Verify Bolmo architecture/PEFT compatibility before a full run.

```
Task 1 input: original_script + translation
Task 1 target: gold_transliteration

Task 2 input: original_script + translation + gold_transliteration
Task 2 target: gold_gloss

Inference: original_script + translation -> predicted_transliteration
           -> predicted_gloss (without gold transliteration)
```

4. LoRA baseline and tunable parameters

Apply the same baseline setup separately to 1B and 7B when hardware allows. If 7B full experiments are infeasible, report the limitation explicitly.

Setting	Proposed baseline / rule
LoRA rank (r)	16
LoRA alpha	32
LoRA dropout	0.05
Learning rate	2e-4 (compare with 1e-4 in a small experiment)
Epochs	3
Evaluation frequency	Every 0.5 epoch if practical; otherwise at least every epoch
Save checkpoint frequency	Match evaluation frequency when storage permits
Effective batch size	Fix a common target after checking GPU memory
Random seed	42
LoRA target modules	Determine from actual Bolmo architecture; validate gradient flow

Learning rate: how strongly trainable weights change during each optimization step. Too high can make training unstable; too low may learn slowly. **Evaluation frequency:** how often the model is evaluated on held-out validation data during training. More frequent evaluation offers more checkpoints but costs computation. Set both consistently and record the values.

5. Evaluation metrics and checkpoint selection

• **Final test metrics:** morpheme-level accuracy and word-level accuracy, with exact definitions/alignment based on Ginn et al. (2024).

• **Validation experiment:** compute teacher-forced validation loss and generated-gloss chrF++ for every saved checkpoint; use a fixed chrF++ configuration.

• **Selection A:** lowest validation loss. **Selection B:** highest validation chrF++. Evaluate the chosen checkpoints on the same held-out test set, using the two main accuracy metrics.

• Validation and test never update model parameters. Keep gold answers out of inference prompts. Score malformed/empty outputs consistently; never silently skip them.

6. Run sequence and deliverables

• 1. Finalize corrected dataset schema and stable train/validation/test split; validate script, translation, transcription, and gloss alignment.

• 2. Smoke test both zero-shot prompts on a few verified examples, then freeze prompt versions.

• 3. Run 1B and 7B zero-shot All-in-one and Step-by-step; save per-example predictions.

• 4. Verify LoRA adapter compatibility; run a small training smoke test, then train the baseline and save checkpoints.

• 5. Compare validation loss vs. chrF++ checkpoint selection and, if resources allow, a small learning-rate/LoRA-parameter sweep.

• 6. Export CSV/JSONL predictions, configuration files, checkpoint metrics, and final result table.

Model / method	Morpheme acc.	Word acc.
Bolmo 1B / zero-shot all-in-one	TBD	TBD
Bolmo 1B / zero-shot step-by-step	TBD	TBD
Bolmo 7B / zero-shot all-in-one	TBD	TBD
Bolmo 7B / zero-shot step-by-step	TBD	TBD
LoRA / loss-selected (per model)	TBD	TBD
LoRA / chrF++-selected (per model)	TBD	TBD

Optional continuation from the agenda: related high-resource languages, extended pan/pnb datasets, and additional low-resource target languages. These are out of scope for the initial Korean-only baseline.

7. Learning-rate and Evaluation-frequency Experiments

Add the following controlled comparisons to the LoRA section of V3. Run them on the corrected Korean dataset only, with identical train/validation/test splits, prompts, seeds, and evaluation rules. These settings are proposed starting points, not established optima.

A. Learning-rate comparison (requires separate training runs)

Keep LoRA rank = 16, alpha = 32, dropout = 0.05, and all other training parameters fixed. Train once per learning rate.

Run	Learning rate	Evaluation frequency	Purpose
LR-1	1e-4	Every 100 optimizer steps	Lower learning rate
LR-2 (baseline)	2e-4	Every 100 optimizer steps	Baseline
LR-3	5e-4	Every 100 optimizer steps	Higher learning rate

B. Evaluation-frequency comparison (can reuse a dense checkpoint run)

Hold the learning rate at 2e-4. Compare candidate evaluation schedules based on a **common saved checkpoint pool**. For example, train and save every 50 optimizer steps, then simulate these schedules by selecting subsets of saved checkpoints.

Run	Evaluation frequency	Learning rate	Checkpoint candidates
EF-1	Every 50 steps	2e-4	50, 100, 150, ...
EF-2 (baseline)	Every 100 steps	2e-4	100, 200, 300, ...
EF-3	Every 200 steps	2e-4	200, 400, 600, ...

Important: 50/100/200 steps are illustrative. Check the total number of optimizer steps before finalizing the intervals. If training is short, use fractions of an epoch instead. Evaluation frequency alone does not change learned weights unless evaluation is coupled to early stopping or other training decisions. If checkpoints are saved only at evaluation time, less frequent evaluation also limits which models can be selected.

C. How to compare the results

At each candidate checkpoint, record validation loss and validation chrF++ (computed from generated glosses). Select the best checkpoint **once by minimum validation loss** and **once by maximum validation chrF++**. Compare the selected checkpoints on the same held-out test set using morpheme-level and word-level accuracy. Also report total training steps, evaluation runtime, and checkpoint count.

Efficiency: Full 3 x 3 combinations are not needed initially. Use 3 learning-rate runs plus an evaluation-frequency analysis from the baseline run. For Bolmo 1B and 7B, report the model size in every row and keep comparisons separate.